



ROBOT SYSTEM DESIGN
LABORATORY

人協働マニピュレーション実装講座(ROS2) Vol.2

アジェンダ

1. Vol.1の振り返り
2. SEED-Noid-Mover動作確認
3. SEED-Noid-Mover開発環境構築
4. ROS2要素技術(URDF/XACRO、msg/srv、launch、ros2_control)
5. SEED-Noid-Moverの動作確認
6. RRI人協働マニピュレーション機能インターフェース仕様
7. 人協働マニピュレーション実装
8. まとめ





ROBOT SYSTEM DESIGN

LABORATORY

Vol.1の振り返り

ROSについて

ROSとは、Robot Operating Systemの略で、ロボット開発のために必要な、以下の機能で成り立っている。

plumbing (通信)

ROSの中核で、ノード間のデータ交換を担う通信基盤。
Topic (Pub/Sub)、Service、Actionなどにより分散処理を実現。
異なるプロセス・マシン間でも柔軟かつ非同期に連携できる。



tools (ツール群)

開発・デバッグ・可視化を支援するための各種ツール。
例としてrviz (可視化) やros2 topicコマンドなどがある。
システムの状態確認や効率的な開発を可能にする。

capabilities (機能群)

ロボット開発に必要な基本機能をパッケージとして提供。
ナビゲーション、マニピュレーション、認識などが含まれる。
これらを組み合わせることで高度なロボット機能を実現できる。

community (コミュニティ)

OSSとしてのコミュニティと再利用可能な資産の集合。
多数のパッケージやドキュメント、ユーザー知見が共有されている。
開発効率を高め、他者の成果を活用できる環境を形成している。

ROS2について



The Robot Operating System (ROS) is a set of software libraries and tools for building robot applications. From drivers and state-of-the-art algorithms to powerful developer tools, ROS has the open source tools you need for your next robotics project.

(引用: <https://docs.ros.org/en/jazzy/index.html>)

ROS2(Robot Operating System 2)は、ロボットアプリケーション開発のための、ソフトウェアライブラリとツール群のこと。(→ミドルウェア)

ドライバや最先端のアルゴリズムから強力な開発ツールに至るまで、ROSにはロボットプロジェクトに必要なオープンソースツールが揃っている。

- ROS1の後継ミドルウェア。
- DDS(Data Distribution Service)を通信基盤として採用している。
- **分散システム・マルチロボット・リアルタイム性**を強化。
- **自律移動・マニピュレーション・センサ処理・位置推定**などの機能がすぐ使える形で提供。

DDSとは：分散システムにおける“データ通信の仕組み(ミドルウェア規格)”です。リアルタイムかつ信頼性の高いデータ共有のための国際標準通信規格です。主に、Pub/Sub通信モデルを使用し、遅延が少ない点と機器間の柔軟な接続性から多くの産業システムで採用されている。



ROS2環境構築

- ロケール設定

```
$ sudo apt update && sudo apt upgrade
```

```
$ sudo apt install locales
```

```
$ sudo locale-gen ja_JP ja_JP.UTF-8
```

```
$ sudo update-locale LC_ALL=ja_JP.UTF-8 LANG=ja_JP.UTF-8
```

```
$ export LANG=ja_JP.UTF-8
```

確認

```
$ locale
```

```
LANG=ja_JP.UTF-8
```

...

- 必要ツールインストール

```
$ sudo apt install -y curl gnupg lsb-release python3-colcon-common-extensions  
python3-rosdep python3-vcstool git software-properties-common python3-pip python3-  
colcon-clean
```



ROS2環境構築

- ROS2 aptリポジトリ追加

```
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

- ROS2 Jazzyインストール

```
$ sudo apt update
```

```
$ sudo apt install ros-jazzy-desktop
```

```
$ sudo apt install -y ros-${ROS_DISTRO}-rqt-graph ros-${ROS_DISTRO}-gazebo-ros-pkgs ros-${ROS_DISTRO}-gazebo-ros2-control ros-${ROS_DISTRO}-ros2-controllers ros-${ROS_DISTRO}-control-test-assets ros-${ROS_DISTRO}-ros2-control
```

ROS2環境構築

- 環境設定(起動時に読み込まれるように設定)

```
$ echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc  
$ source ~/.bashrc
```

- rosdep初期化

```
$ sudo rosdep init  
$ rosdep update
```

- パッケージインストール

```
$ sudo apt install ros-${ROS_DISTRO}-nav2-bringup  
ros-${ROS_DISTRO}-laser-proc  
ros-${ROS_DISTRO}-laser-filters  
ros-${ROS_DISTRO}-ros2-control-cmake  
ros-${ROS_DISTRO}-tf-transformations  
ros-${ROS_DISTRO}-moveit  
ros-${ROS_DISTRO}-moveit-ros-planning-interface  
ros-${ROS_DISTRO}-moveit-core  
ros-${ROS_DISTRO}-moveit-common  
ros-${ROS_DISTRO}-moveit-ros-planning  
ros-${ROS_DISTRO}-moveit-visual-tools  
mplayer  
ros-${ROS_DISTRO}-joy-linux  
ros-${ROS_DISTRO}-joy-linux-dbgsym
```



ROS2環境構築

- 動作確認

talker/lintenerテスト

(ターミナル1)

```
$ ros2 run demo_nodes_cpp listener
```

(ターミナル2)

```
$ ros2 run demo_nodes_cpp talker
```

```
rsdlab@rsdlab-NUC11TNKi3:~$ ros2 run demo_nodes_cpp listener
```

```
rsdlab@rsdlab-NUC11TNKi3:~$ ros2 run demo_nodes_cpp talker
```

- ワークスペース作成(seed_robot_ros2_pkg)

```
$ mkdir -p ~/colcon_ws/src
```

```
$ cd ~/colcon_ws/src
```

```
$ git clone --recurse-submodules https://github.com/thkrrc1/seed_robot_ros2_pkg.git
```

- パッチ適用

```
$ cd ~/colcon_ws/src/seed_robot_ros2_pkg
```

```
$ patch -p0 < patch/urg_node2.patch
```





ROBOT SYSTEM DESIGN

LABORATORY

SEED-NoiD-Mover

環境構築～動作確認

SEED-Noïd-Mover開発環境構築

- THK株式会社が開発した、SEED-Noïd/SEED-Lifter/SEED-Moverの3つのユニットからなるロボット
- SEED-Noïd
上体ヒューマノイド。頭部・双腕を担う部分。
- SEED-Lifter
昇降ユニット。搬送物や上体の高さ調整を担う部分。
- SEED-Mover
全方向移動台車。メカナムホイールを使用して移動する部分。
360°旋回が可能。

SEED-Noïd

SEED-Lifter

SEED-Mover



SEED-NoId-Mover開発環境構築

- ワークスペース作成(seed_robot_ros2_pkg)

```
$ mkdir -p ~/seed_ws/src
```

```
$ cd ~/seed_ws/src
```

```
$ git clone -b https://github.com/rsdlab/seed_robot_ros2_pkg.git
```

- パッチ適用

```
$ cd ~/seed_ws/src/seed_robot_ros2_pkg
```

```
$ patch -p0 < patch/urg_node2.patch
```



SEED-NoId-Mover開発環境構築

- ロボットプロジェクトのクローン

```
$ cd ~/seed_ws/src/seed_robot_ros2_pkg/robots
```

```
$ git clone -b ros2-jazzy
```

```
https://github.com/rsdlab/noid\_lifter\_mover.git
```

- ビルド

```
$ cd ~/seed_ws
```

```
$ colcon build --symlink-install
```

```
$ source install/setup.bash
```



SEED-Noi-Mover開発環境構築

- Udev設定(実機を動かす場合)

ロボットのUSBをPCにまだ登録していない場合は、登録する必要があります。

(/etc/udev/rules.d/90-aero.rulesがすでにある場合、こちらの作業は必要ありません。)

スクリプトの実行

```
$ cd ~/seed_ws/src/seed_robot_ros2_pkg/scripts
```

```
$ ./make_udev_install.sh
```

実行すると下記メッセージが表示されます。

udevファイルをコピーします

完了しました

SEED-Noïd-Mover開発環境構築

- Gazebo用パッケージをクローン

```
$ cd ~/seed_ws/src
```

```
$ git clone https://github.com/ros-controls/gz\_ros2\_control.git
```

```
$ touch gz_ros2_control/gz_ros2_control_demos/COLCON_IGNORE
```

- 移動機能パッケージをクローン

```
$ cd ~/seed_ws/src
```

```
$ git clone -b ros2_jazzy https://github.com/rsdlab/movement_function.git
```



SEED-Noid-Mover開発環境構築

- Githubから人協働マニピュレーションパッケージをクローン

```
$ cd ~/
```

```
$ git clone -b ros2-jazzy
```

```
https://github.com/rsdlab/Human\_Collaboraiton\_System.git
```

```
$ cd Human_Collaboration_System/
```

```
$ cp Seed-noid/* ~/seed_ws/src
```



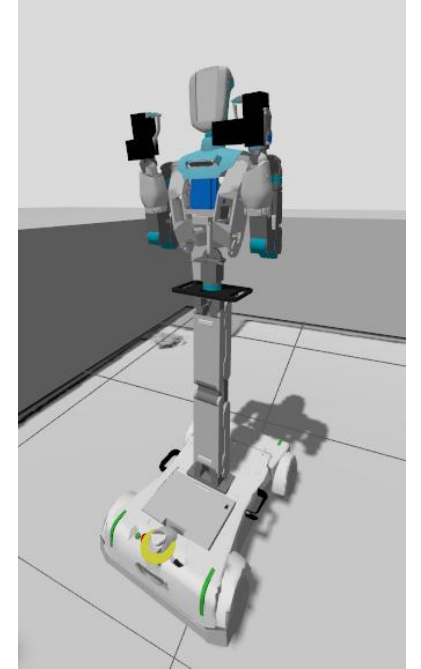
URDF / XACRO

URDF (Unified Robot Description Format) とは

- ・ ロボットの構造（リンク・関節）をXMLで記述するフォーマット
- ・ RViz2・Gazebo・MoveIt2 でのロボット表示と制御の基盤
- ・ 定義要素：link（剛体）、joint（接続・可動域）、material（外観）

XACRO（マクロ拡張）

- ・ URDFをマクロ・変数・条件分岐で記述できる拡張形式
- ・ 重複コードを削減し、複数モデルのパラメータ化が容易
- ・ SEED-R7 パッケージ内の *.urdf.xacro ファイルがロボット定義の起点



(Gazebo上でSEED-Noidを表示)

msg / srv / action ファイル

msgファイル(メッセージ定義)とは

- ・ トピック通信で送受信するデータ構造を定義するファイル(拡張子 .msg)
- ・ 1行 = 「型 フィールド名」で記述(例: float64 x / string name)
- ・ ビルド時にPython/C++のクラスへ自動生成され、Publisher/Subscriberで利用

srvファイル(サービス定義)とは

- ・ サービス通信(要求/応答)のインタフェースを定義するファイル(拡張子 .srv)
- ・ 「---」を境に上=Request(要求)、下=Response(応答)を記述
- ・ CMakeLists/package.xmlに rosidl_generate_interfaces を登録してビルド

actionファイル(アクション定義)とは

- ・ 長時間処理(Goal送信→Feedback配信→Result返却)のインタフェースを定義するファイル(拡張子 .action)
- ・ 「---」を2つ用い、上から Goal(目標)/ Result(結果)/ Feedback(途中経過)の3部構成で記述
- ・ 内部で複数の msg / srv が自動生成され、Action Server/Client で利用



launchファイル

launchファイルとは

- 複数のノード・パラメータ・名前空間を一括起動するROS2の仕組み
- Python形式 (*.launch.py) が現在の標準。XMLおよびYAML形式も利用可能
- 起動コマンド : `ros2 launch <package> <launch_file>`

主な構成要素

- Node : 起動するノードをパッケージ名・実行ファイル名で指定
- DeclareLaunchArgument : 引数を宣言しデフォルト値を設定
- IncludeLaunchDescription : 別のlaunchファイルを入れ子で呼び出し
- SetParameter / SetParametersFromFile : YAMLでパラメータを一括注入



ros2_control / Controller Manager

ros2_control とは

- ・ ROS2標準のハードウェア抽象化フレームワーク
- ・ 実機・シミュレーションを同一インタフェースで切り替え可能
- ・ 構成 : Hardware Interface → Controller Manager → Controller

主なコントローラ

- ・ joint_trajectory_controller : MoveIt2から軌道を受け取りマニピュレータを駆動
- ・ joint_state_broadcaster : 関節状態を /joint_states トピックに配信
- ・ forward_position_controller : 位置指令を直接ハードウェアへ送信
- ・ 設定はURDF内の <ros2_control> タグおよびYAMLパラメータファイルで管理



Seed-Lifter-Moverの動作確認

- Gazeboを使用して動作確認

```
$ ros2 launch noid_lifter_mover bringup_gazebo.launch.py
```

```
$ ros2 run map_management map_management_node
```

```
$ ros2 run node_transformation get_environment_coordinates_server
```

```
$ ros2 run workpieces_detection_subsystem WorkpiecesDetectionNode
```

```
$ ros2 run place_position_detection_subsystem
```

```
PlacePositionDetectionNode
```

```
$ ros2 run collaboration_manipulation_module
```

```
CollaborationManipulationModule --ros-args -p use_sim_time:=true
```

```
$ ros2 run mobile_robot_navigation_module
```

```
mobile_robot_navigation_node --ros-args -p use_sim_time:=true
```

```
$ ros2 run external_app pose_hint_client
```

```
$ ros2 run management_system ManagementSystemNode
```

実機で動かした様子



RRI人協働マニピュレーション機能インターフェース仕様について

- ロボット革命・産業IoTイニシアティブ協議会(RRI)により定義された人協働マニピュレータのためのインターフェース仕様書
- 状態遷移や外部からの命令など、システムの中で機種を問われない部分を定義する。
- 機種依存部のみを実装することで、命令部分を変更することなく多様な機種で同じ作業を行うことが可能。

資料：<https://www.jmfrri.gr.jp/library/library-6004/>



RRI人協働マニピュレーション機能インターフェース仕様について

• 従来

- 少品種大量生産が前提
- コントローラのIFで周辺装置と密に連携
- 事前プログラムした動作を正確に反復
- 安全のため人とロボットを空間的に分離



• これから

- 三品産業・物流へ適用拡大、多品種少量生産化
- 密連携の運用がコストに見合わなくなる
- 人が適宜介入する運用が現実的な選択肢に
- 同空間で協働する人協働ロボットの普及



RRI人協働マニピュレーション機能インターフェース仕様について

仕様書のスコープ

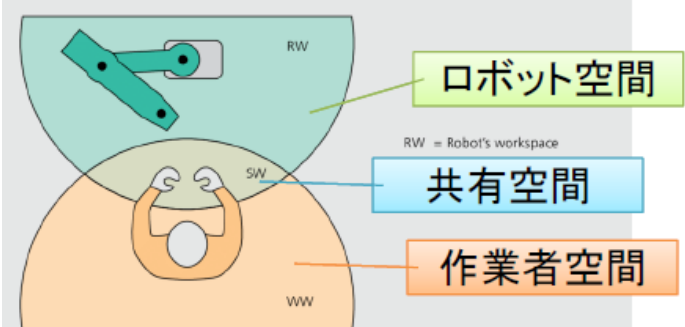
“人協働”を広義にとらえ、人が適宜介入できる仕組みまで含めて規定

狭義の“人に安全な協働ロボット”に留まらず、ロボット単独運用のハンドルに人が介入する“広義の人協働マニピュレーション”を対象する

01. ユースケース	02. システム構成要素	03. ソフトウェアIF
人協働マニピュレーションシステムで想定される利用シーン。運用シナリオを定義	システムを成り立たせる構成要素(モジュール)を整理して、役割を明確化	構成要素間をつなぐソフトウェアインターフェースを定義して、責任分担を容易に。



RRI人協働マニピュレーション機能インターフェース仕様について



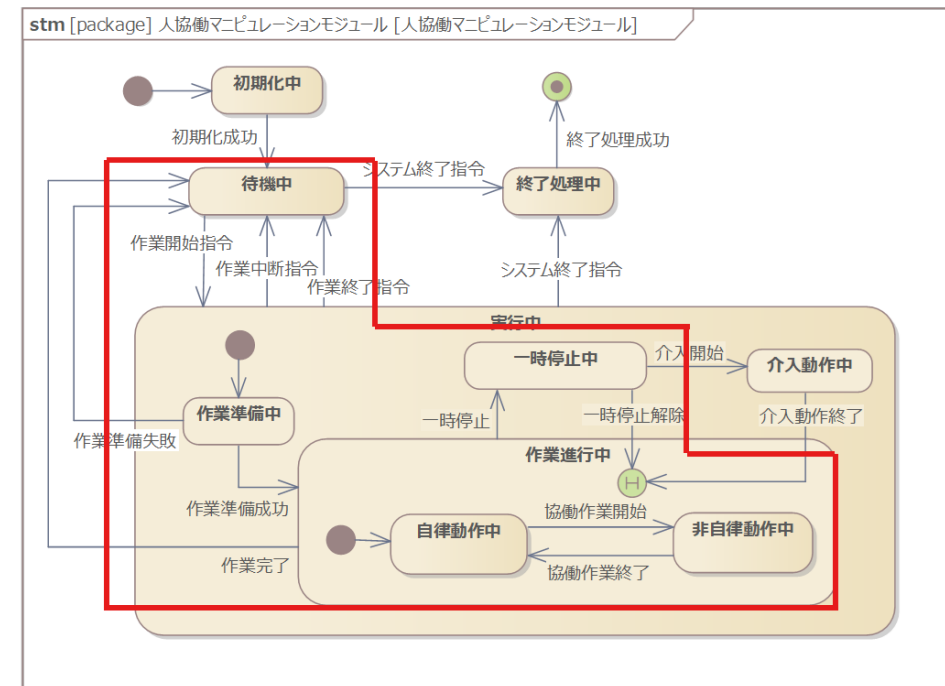
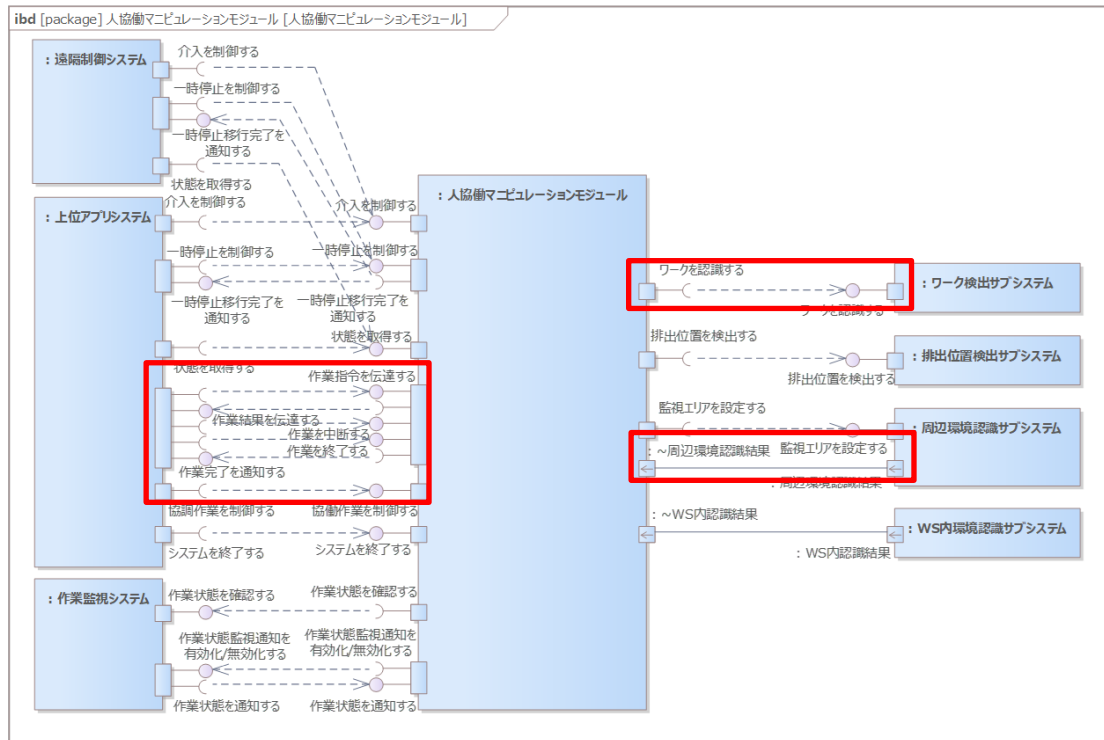
	Cell	Coexistence	Synchronized	Cooperation	Collaboration
説明	防護柵前提の産ロボ状態	防護柵なし	協働作業		
空間共有	隔離	なし	空間共有, 時間的に分離	時間, 空間的に共有	接触による相互作用あり
時間	制限なし	なし	交代	同時	同時

引用元: https://www.researchgate.net/figure/The-various-levels-of-cooperation-between-a-human-worker-and-a-robot_fig2_327744724



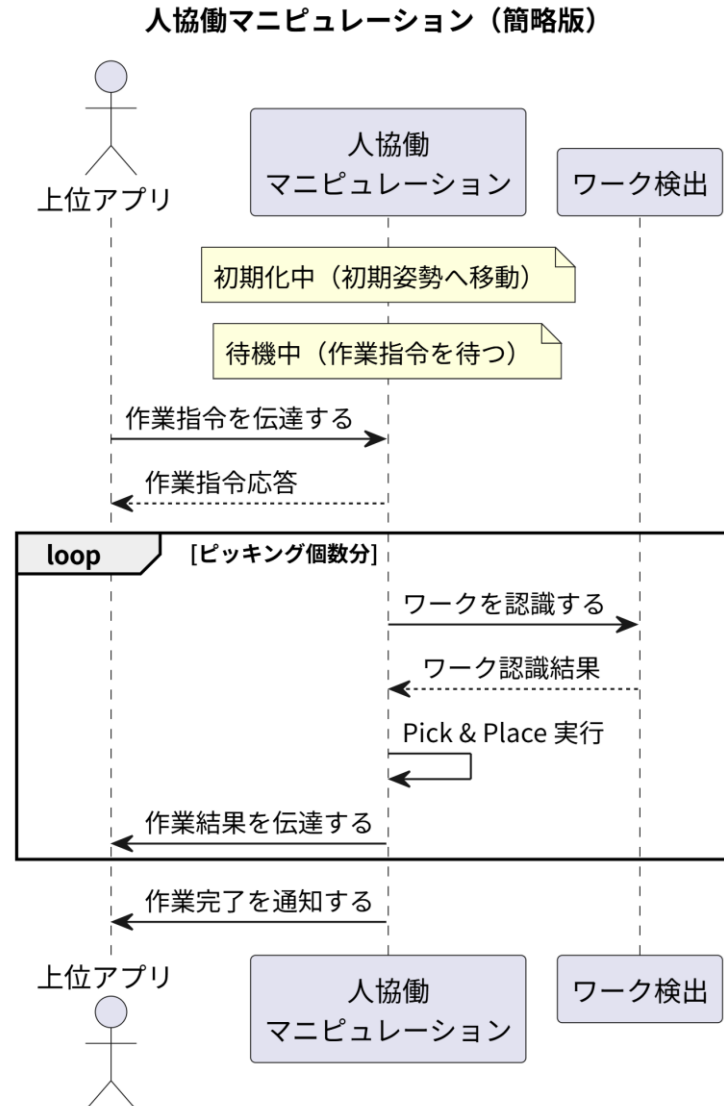
RRI人協働マニピュレーション機能インターフェース仕様について

- 外部インターフェース図・人協働マニピュレーションモジュールの状態遷移図で今回取り扱う範囲(赤枠)



RRI人協働マニピュレーション機能インターフェース仕様について

人協働マニピュレーションのシーケンス図は以下のようになっている





ROBOT SYSTEM DESIGN
LABORATORY

人協働マニピュレーションモジュール の実装

人協働マニピュレーションのパッケージ構成

	パッケージ名	説明
collaboration_manipulation	└ collaboration_manipulation_message	msg / srv のインタフェース定義
	└ collaboration_manipulation_module	協働動作のモジュール群・状態遷移
	└ management_system	タスク全体を統括する管理ノード
	└ peripheral_environment_detection_subsystem	周辺環境・人検出
	└ place_position_detection_subsystem	設置位置の検出
	└ task_status_monitor_system	タスク状態の監視
	└ teleopeletion_system	人による遠隔操作・介入
	└ workpieces_detection_subsystem	ワーク(対象物)検出
	└ workspace_perception_subsystem	作業空間の認識

周辺環境認識サブシステム->マニピュレーション実装

- 周辺環境・人検出

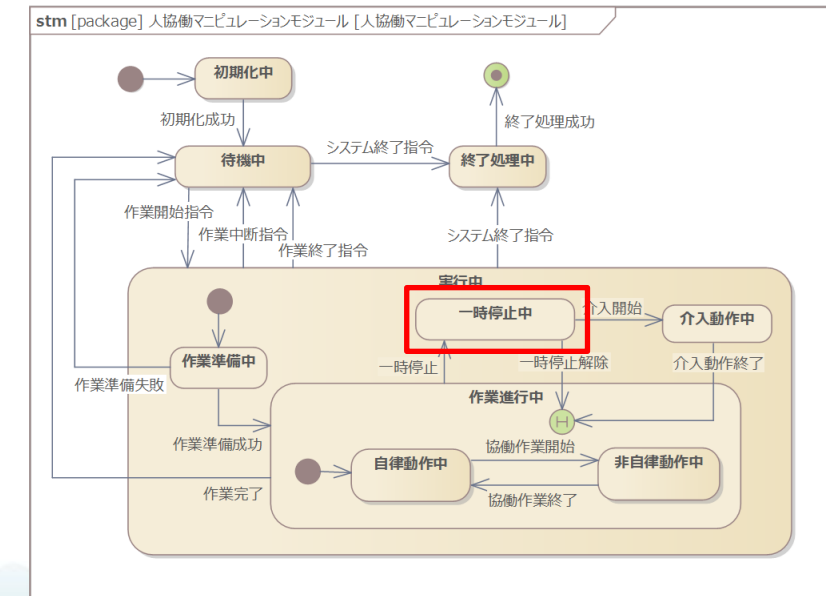
ファイルの場所：

collaboration_manipulation/peripheral_environment_detection_subsystem/peripheral_environment_detection_subsystem/PerEnvDetectNode.py

人を検知するとジョブを一時停止する。

一時停止後、人が検知されなくなると、一時停止を解除し再度ジョブが実行される。

intrusion_detection_exercise/内のファイルを実装し、動作確認する。



マニピュレーション->ワーク検出実装

- ワーク検出サービスサーバ

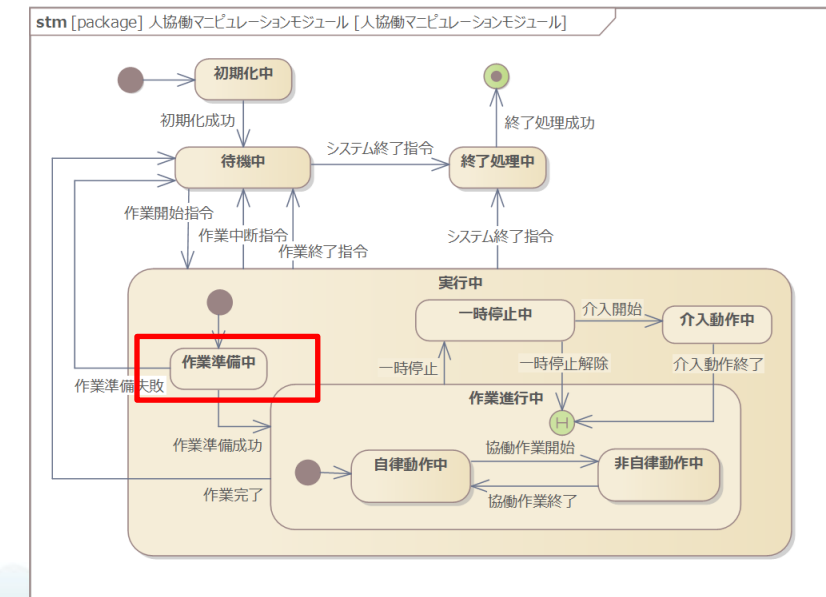
ファイルの場所：

collaboration_manipulation/workpieces_detection_subsystem/workpieces_detection_subsystem/WorkpiecesDetectionNode.py

作業の準備段階で"対象エリアにどんなワークがあるか"をワーク認識サブシステムに問い合わせる。

workpieces_detection_exercise/内のファイルを実装し、動作確認する。

[WARN] [1781661036.208017296] [move_planner]: Human detected (/intrusion_result=1). Pausing before next motion...



上位アプリ -> マニピュレーション実装

- 上位アプリから作業指示を指令

ファイルの場所：

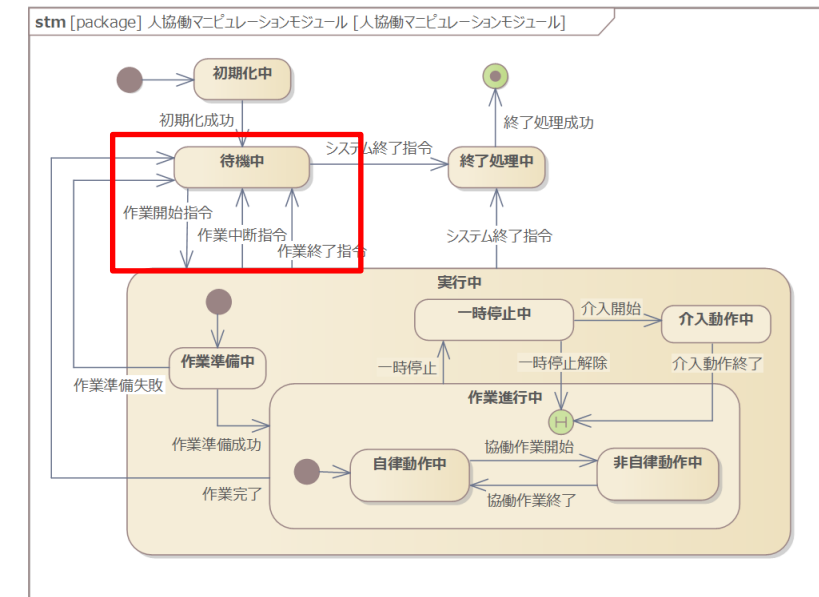
collaboration_manipulation/management_system/management_system/ManagementSystemNode.py

ROS2サービス"send_command_serive"を呼んで、人協働マニピュレーションモジュールにPICKコマンドを送信する。

management_system_exercise/内のファイルを実装し、動作確認する。

Request Data:

```
collaboration_manipulation_message.srv.DetectWorkpieces_Request(task_command_id=1, work_type_id=5, target_area=collaboration_manipulation_message.msg.TargetArea(start_point=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0), end_point=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0)), properties=[])
```





ROBOT SYSTEM DESIGN

LABORATORY

まとめ

今回の講座を通して

「ROS2の基礎」から「実機での人協働システム実装」まで、
理論と実装を一貫して学びました。

本教材で得た知識と実装経験を土台に、より複雑な人協働シナリオ
や実運用への応用に挑戦してみてください。





ROBOT SYSTEM DESIGN

LABORATORY

参考文献

参考文献

- ROS2 Docs:
<https://docs.ros.org/>
- Gazebo Docs:
<https://gazebo-sim.org/docs/harmonic/getstarted/>
- Nav2 Docs:
<https://docs.nav2.org/>
- MoveIt2:
<https://moveit.picknik.ai/main/index.html>
- ROS2とPythonで作って学ぶAIロボット入門 改訂第2版
(出村・萩原・升谷・タン著, 講談社)

